

Abstimmung

26.06.2025

Heute zu besprechen:

Simulation Class Methods



plot_outputs()



monitor_results()

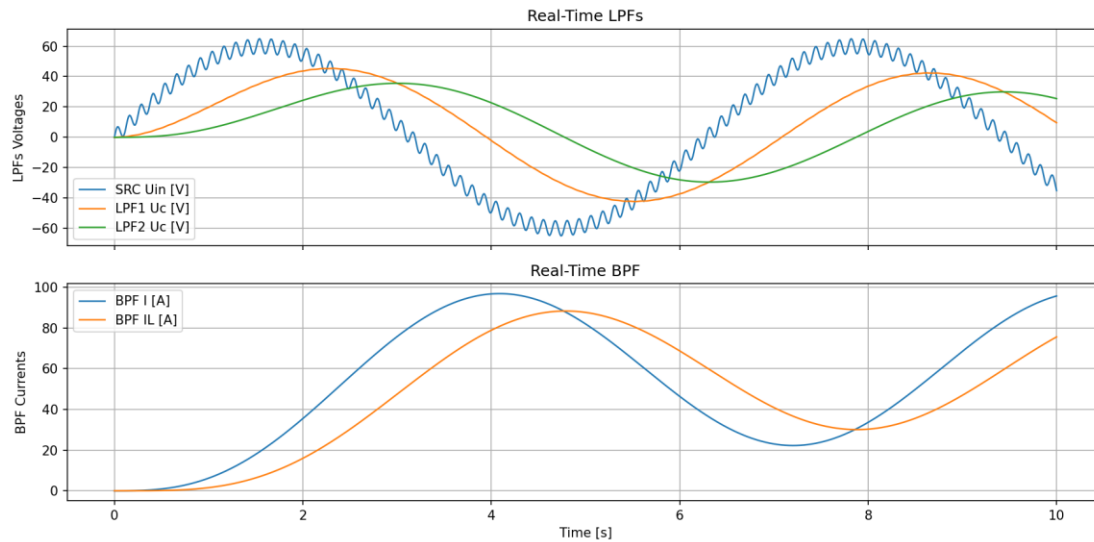


export_history_to_excel()



visualise()

main() & plot_outputs()



```
# Create modules
src = Module("SRC", "signal_source.py")
lpf1 = Module("LPF1", "lowpass_filter.py", parameters={"R": (1000, "ohm"), "C": (1, "mF")}, normalise_units=True)
lpf2 = Module("LPF2", "lowpass_filter.py", parameters={"R": (1000, "ohm"), "C": (0.001, "F")})
bpf = Module("BPF", "bandpass_filter.py", parameters={"R": (1.26, "ohm"), "L": (1, "H"), "C": (0.0253, "F")}, dt=0.02)

# Create simulation
sim = Simulation(modules=[bpf, src, lpf2, lpf1], dt=0.01, t_end=1) # Module order will be determined automatically

# Define connections
sim.connect("SRC", "Uin", "LPF1", "Uin")
sim.connect("LPF1", "Uc", "LPF2", "Uin")
sim.connect("LPF2", "Uc", "BPF", "U")

# Plotting configuration
plot_config={
    "groups": [
        [("SRC", "Uin"), ("LPF1", "Uc"), ("LPF2", "Uc")], # list -> one subplot, tuple -> pair to determine output time series
        [("BPF", "I"), ("BPF", "IL")]
    ],
    "titles": ["Real-Time LPFs", "Real-Time BPF"], # titles per subgraph
    "y_labels": ["LPFs Voltages", "BPF Currents"] # labels per subgraph
}

# Run simulation

sim.run()

sim.run(monитор_results=True, monitor_config=plot_config)

sim.visualise("electrical_circuit", show_edge_labels=True)

sim.plot_outputs(config=plot_config)

sim.export_history_to_excel(outputs=[("BPF", "I"), ("SRC", "Uin")], time_range=(1, 1.5))

sim.export_history_to_excel()
```

`sim.run(monitor_results=True, monitor_config=plot_config)`

The screenshot displays the Spyder Python IDE interface. The main editor window shows a Python script for simulating an electrical circuit. The script defines modules for a signal source, two lowpass filters, and a bandpass filter, connects them, and runs a simulation with monitoring enabled. The script also configures plotting and exports the simulation history to an Excel file.

```
4 t0 = time.time()
5
6
7 # Create modules
8 src = Module("SRC", "signal_source.py")
9 lpf1 = Module("LPF1", "lowpass_filter.py", parameters={"R": (1000, "ohm"), "C": (0.001, "F")})
10 lpf2 = Module("LPF2", "lowpass_filter.py", parameters={"R": (1000, "ohm"), "C": (0.001, "F")})
11 bpf = Module("BPF", "bandpass_filter.py", parameters={"R": (1.26, "ohm"), "L": (1, "H"), "C": (0.0253, "F"), dt=0.02})
12
13 # Create simulation
14 sim = Simulation(modules=[bpf, src, lpf2, lpf1], dt=0.01, t_end=10) # Module order will be determined automatically
15
16 # Define connections
17 sim.connect("SRC", "Uin", "LPF1", "Uin")
18 sim.connect("LPF1", "Uc", "LPF2", "Uin")
19 sim.connect("LPF2", "Uc", "BPF", "U")
20
21 # Plotting configuration
22 plot_config={
23     "groups": [
24         [("SRC", "Uin"), ("LPF1", "Uc"), ("LPF2", "Uc")], # list -> one subplot, tuple -> pair to determine output time series
25         [("BPF", "I"), ("BPF", "IL")]
26     ],
27     "titles": ["Real-Time LPFs", "Real-Time BPF"], # titles per subgraph
28     "y_labels": ["LPFs Voltages", "BPF Currents"] # labels per subgraph
29 }
30
31 # Run simulation
32
33 #sim.run()
34
35 sim.run(monitor_results=True, monitor_config=plot_config)
36
37 sim.visualise("electrical_circuit", show_edge_labels=True)
38
39 #sim.plot_outputs(config=plot_config)
40
41 #sim.export_history_to_excel(outputs=[("BPF", "I"), ("SRC", "Uin")], time_range=(1, 1.5))
42
43 #sim.export_history_to_excel()
44
45 t1 = time.time()
46 print(f"Execution took {t1 - t0:.4f} seconds")
47
48
49
```

The right sidebar shows the 'Variables' panel with the message 'Keine anzuzeigenden Variablen' (No variables to display). Below it, the 'Console' panel shows the IPython 8.27.0 prompt and the execution of the script, which completed successfully.

Python 3.12.7 | packaged by Anaconda, Inc. | (main, Oct 4 2024, 13:17:27) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.
IPython 8.27.0 -- An enhanced Interactive Python. Type '?' for help.
In [1]:

Warnung zur matplotlib backend

- Die Methoden `plot_outputs()` und `_monitor_results()` müssen **unterschiedliche Backends** derselben Plot-Bibliothek verwenden.
- Wenn beide "TkAgg" verwenden (erforderlich für `_monitor_results()`), kann dies zu Abstürzen von `plot_outputs()` führen – **dies ist ein Spyder-Editor-Fehler.**
- Wenn sie unterschiedliche Backends verwenden, wird das Plot-Fenster von `_monitor_results()` geschlossen, bevor `plot_outputs()` ausgeführt wird – **die optimale Lösung bei Verwendung beider Methoden (die Plots sind sowieso gleich)**
- **Wenn nur eine Methode verwendet wird, gibt es kein Problem.**

```
def plot_outputs(self, config, matplotlib_backend="inline"):

    matplotlib.use(matplotlib_backend) # Change backend for plotting to avoid crashes
    # ..... code .....

def _monitor_results(self, config, matplotlib_backend="TkAgg"):

    matplotlib.use(matplotlib_backend) # Change backend for plotting to avoid crashes
    # ..... code .....
```

sim.export_history_to_excel()

- `sim.export_history_to_excel()` gibt alle Outputs über die ganze Zeitspanne
- Alle Module haben gleiches `dt` -> ein Blatt

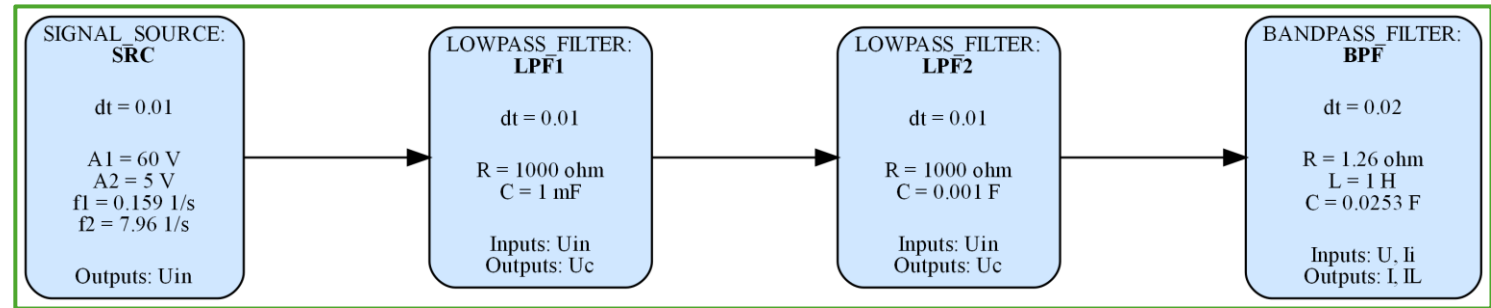
	A	B	C	D	E	F
1	Time [s]	SRC.Uin [V]	LPF1.Uc [V]	LPF2.Uc [V]	BPF.I [A]	BPF.II [A]
2	0	0	0	0	0	0
3	0,01	2,997154688	0	0	0	0
4	0,02	5,406871547	0,029822433	0	0	0
5	0,03	6,785603111	0,083325195	0,000296725	0	0
6	0,04	6,942333722	0,150013455	0,00112286	0,00098918	2,96725E-06
7	0,05	5,98535811	0,217595768	0,002604328	0,00299548	1,41958E-05
8	0,06	4,295737729	0,274980344	0,004743486	0,00585528	4,02391E-05
9	0,07	2,43393726	0,314978106	0,007432291	0,00926441	8,7674E-05
10	0,08	1,002512604	0,336051721	0,010492233	0,01286332	0,000161997
11	0,09	0,49848777	0,342677952	0,01373127	0,01633574	0,000266919
12	0,1	1,19158399	0,34422675	0,017003812	0,01949683	0,000404232
13	0,11	3,058131343	0,352648658	0,020258959	0,02234889	0,00057427
14	0,12	5,786852608	0,379547788	0,023565252	0,02509092	0,00077686
15	0,13	8,85510247	0,433330992	0,027106037	0,02808001	0,001012512
16	0,14	11,65688571	0,517109743	0,031146505	0,03175449	0,001283573
17	0,15	13,6512937	0,627930963	0,035980169	0,03653687	0,001595038
18	0,16	14,49499661	0,75488436	0,041868208	0,0427397	0,001954839
19	0,17	14,12633461	0,8941418	0,048986505	0,050499	0,002373521
20	0,18	12,78040829	1,025752091	0,057393321	0,05975086	0,002863386
21	0,19	10,93147142	1,142639818	0,067025509	0,07025682	0,00343732
22	0,2	9,176738325	1,239941941	0,077724204	0,08167186	0,004107575
23	0,21	8,090069493	1,318791134	0,089283708	0,09363839	0,004884817
24	0,22	8,081378263	1,386024236	0,101511632	0,10588329	0,005777654
25	0,23	9,296199748	1,452487575	0,114285544	0,11829421	0,00679277
26	0,24	11,5800247	1,530367271	0,127592194	0,13095643	0,007935626
27	0,25	14,51613545	1,630195056	0,141539781	0,14414102	0,009211548
28	0,26	17,52767525	1,758247469	0,156340282	0,1582475	0,010626945
29	0,27	20,01894702	1,914990341	0,172266049	0,17371521	0,012190348
30	0,28	21,52132744	2,094952506	0,18959134	0,19092428	0,013913009
31	0,29	21,80805178	2,288051998	0,208533176	0,21011122	0,015808922
32	0,3	20,94975047	2,482049289	0,229205999	0,23131962	0,017894254
33	0,31	19,29713267	2,665528532	0,251601212	0,25439808	0,020186314
34	0,32	17,39505883	2,830678453	0,275596608	0,27904571	0,022702326
35	0,33	15,84905189	2,975235922	0,3009934	0,30489411	0,025458292
36	0,34	15,17694802	3,103001168	0,327572225	0,33160576	0,028468226
37	0,35	15,68202811	3,222812602	0,355153735	0,35896633	0,031743948
38	0,36	17,37870901	3,346443515	0,383648384	0,38694475	0,035295486
39	0,37	19,98899805	3,485696042	0,413084785	0,41570628	0,039131969
40	0,38	23,0105793	3,649488145	0,443608835	0,44558198	0,043262817
41	0,39	25,8398526	3,84165405	0,475453825	0,47699526	0,047698906
42	0,4	27,91978416	4,059997045	0,508888909	0,51036557	0,052453444
43	0,41	28,87634994	4,296824122	0,544158646	0,54601318	0,057542333

- `sim.export_history_to_excel(outputs=`
[("BPF", "I"), ("SRC", "Uin")],
`time_range=(1.2, 1.5))`
- Module haben unterschiedliche `dt`
-> mehrere Blätter

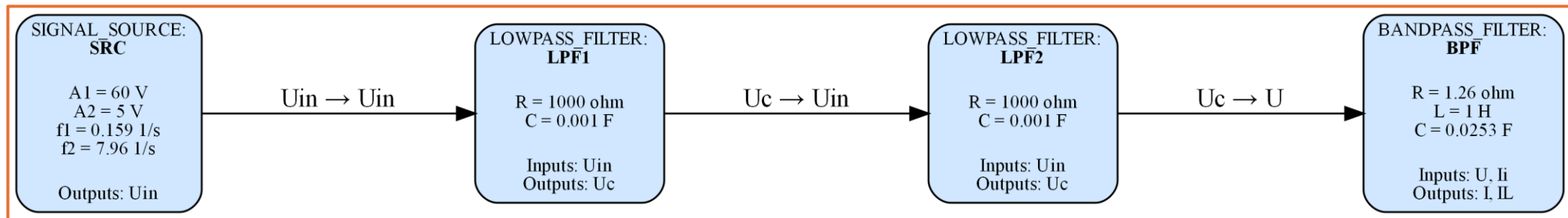
	A	B	C
1	Time [s]	SRC.Uin [V]	
2	1,2	54,29218963	
3	1,21	52,43290272	
4	1,22	51,46927902	
5	1,23	51,68866922	
6	1,24	53,08724391	
7	1,25	55,37092671	
8	1,26	58,02742944	
9	1,27	60,45174306	
10	1,28	62,09434244	
11	1,29	62,59579954	
12	1,3	61,87482834	
13	1,31	60,14819347	
14	1,32	57,87760516	
15	1,33	55,65660999	
16	1,34	54,06518564	
17	1,35	53,52765988	
18	1,36	54,20875923	
19	1,37	55,97325253	
20	1,38	58,41907545	
21	1,39	60,97582143	
22	1,4	63,04447164	
23	1,41	64,14413427	
24	1,42	64,02984635	
25	1,43	62,75257998	
26	1,44	60,64675342	
27	1,45	58,24830629	
28	1,46	56,16340807	
29	1,47	54,91996257	
30	1,48	54,83828527	
31	1,49	55,95263387	
32	1,5	58,00281475	
33			
34			
35			
36			
37			
38			
39			
40			
41			
42			
43			

sim.visualise()

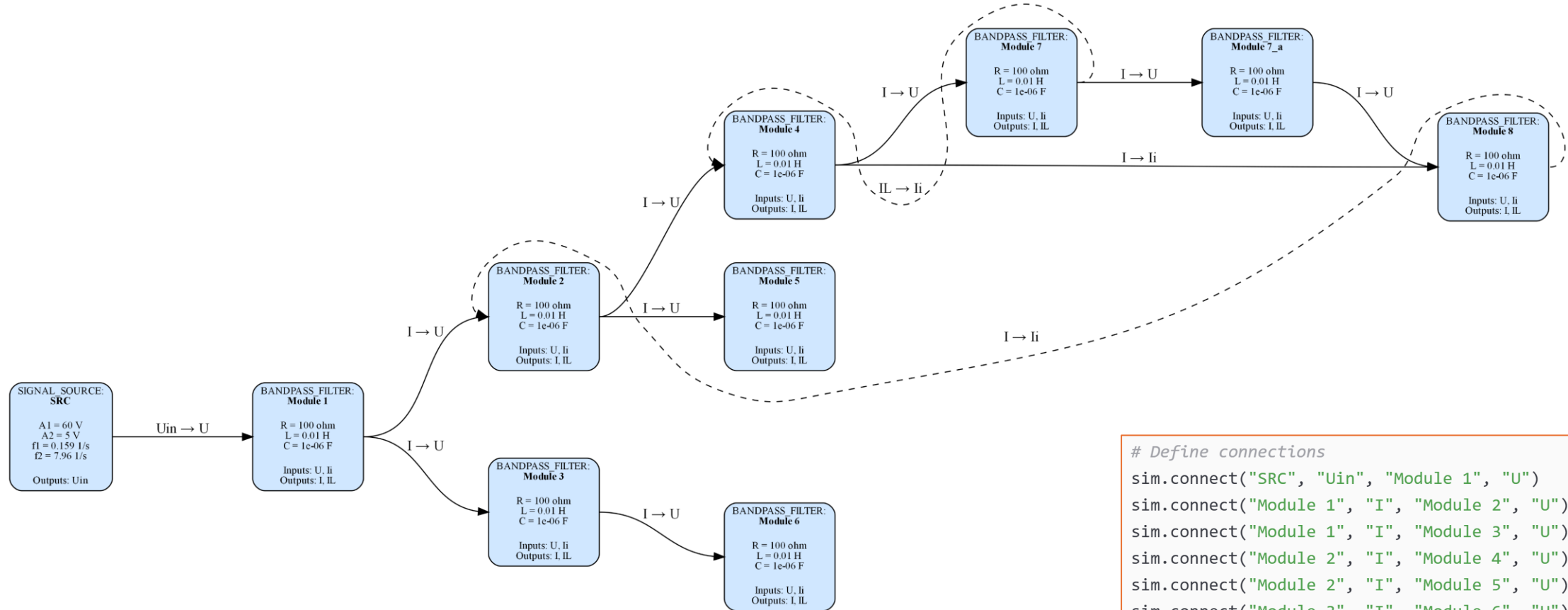
- `show_edge_labels=False`
- **unterschiedliche** Zeitschritte



- `show_edge_labels=True`
- **gleiche** Zeitschritte



sim.visualise()



Define connections

```
sim.connect("SRC", "Uin", "Module 1", "U")
sim.connect("Module 1", "I", "Module 2", "U")
sim.connect("Module 1", "I", "Module 3", "U")
sim.connect("Module 2", "I", "Module 4", "U")
sim.connect("Module 2", "I", "Module 5", "U")
sim.connect("Module 3", "I", "Module 6", "U")
sim.connect("Module 4", "I", "Module 7", "U")
sim.connect("Module 7", "I", "Module 7_a", "U")
sim.connect("Module 7_a", "I", "Module 8", "U")
sim.connect("Module 4", "I", "Module 8", "Ii")
sim.connect("Module 8", "I", "Module 2", "Ii")
sim.connect("Module 7", "IL", "Module 4", "Ii")
```




Danke!